CSC3002
C/C++ Programming

Vector & Stack

Prof. Dr. Ruizhi Chen， SDS

Office: TxC 717

E-mail: chenruizhi@cuhk.edu.cn

1 The collection classes

2 The **Vector** class

3 The **Stack** class

4 Iterating over a collection

Introduction to the C++ Standard Libraries

- A collection of ***classes*** and ***functions***, which are written in the core language and part of the C++ ISO Standard itself. Features of the C++ Standard Library are declared within the `std` ***namespace***
  - Containers: vector, queue, stack, map, set, etc.
  - General: algorithm, functional, iterator, memory, etc.
  - Strings
  - Streams and Input/Output: iostream, fstream, sstream, etc.
  - Localization
  - Language support
  - Thread support library
  - Numerics library
  - C standard library: cmath, cctype, cstring, cstdio, cstdlib, etc.

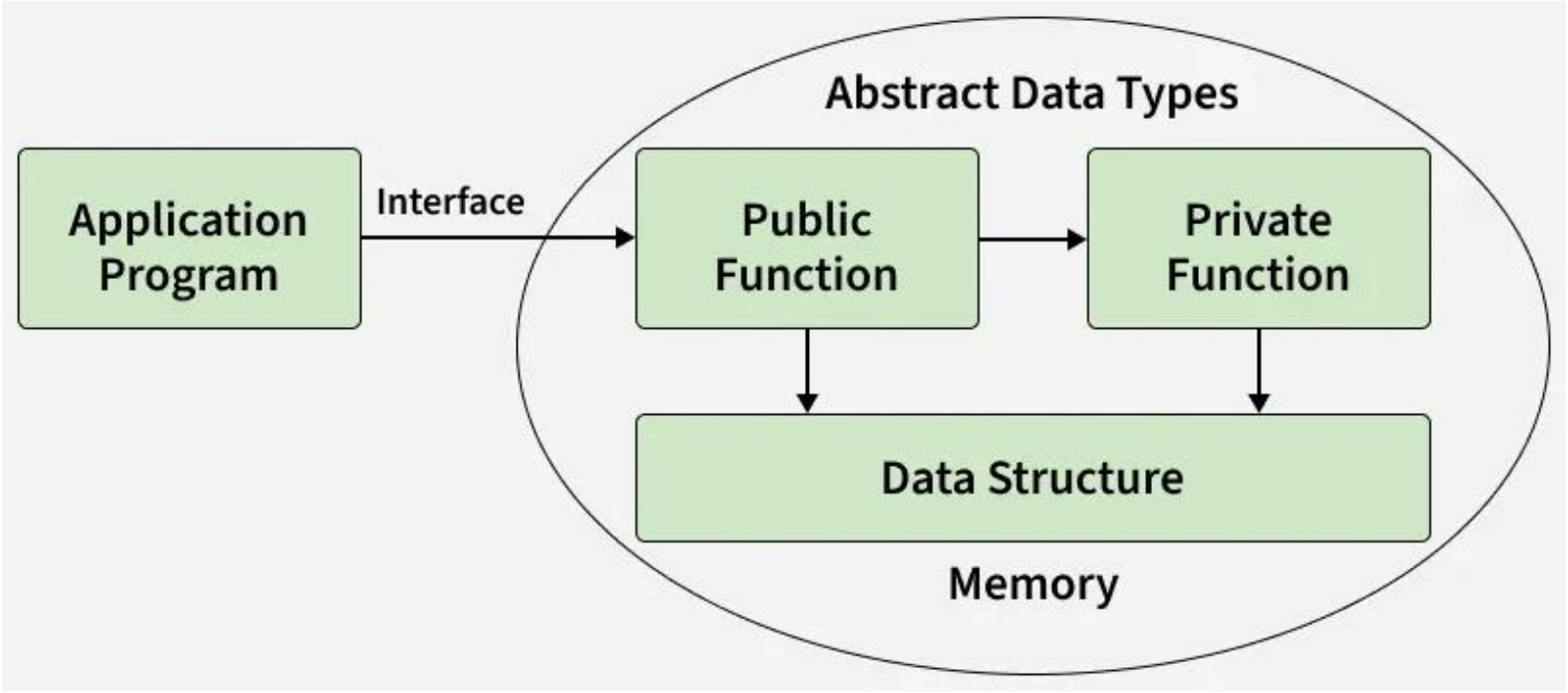
The *Standard* Collection Classes

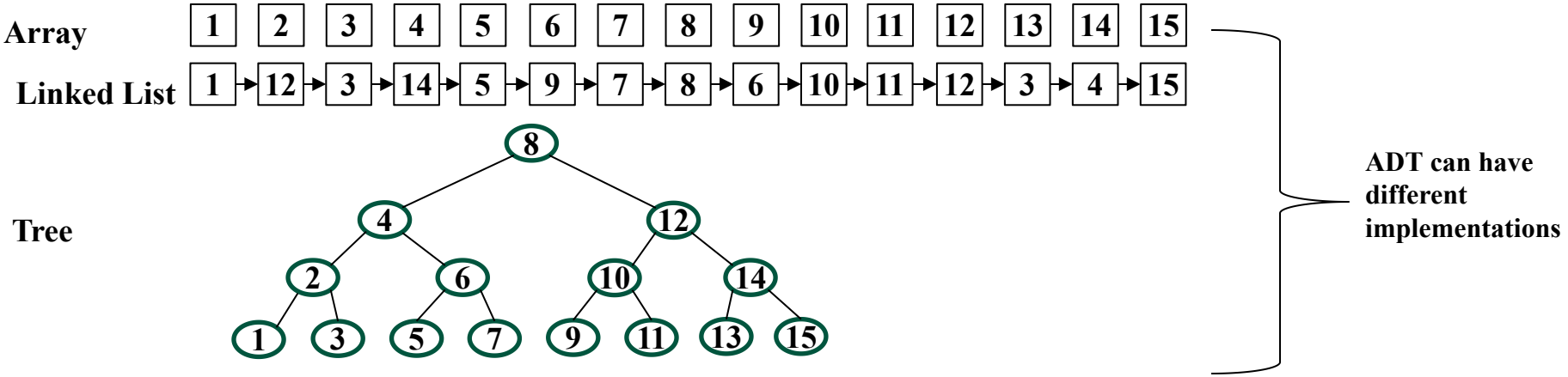
- Classes that include a base-type specification are called ***parameterized classes***.
- In C++, parameterized containers are implemented as ***template*** classes, which make it possible for an entire family of classes to share the same code, although their base types are different.
- The Standard Template Library (STL) is a software library for the C++ programming language that influenced many other parts of the C++ Standard Library. STL is also part of the Standard Library.
- Templates are sometimes called static (or compile-time) ***polymorphism***, as opposed to run-time polymorphism.
- For the moment, you don't need to understand how these classes are implemented using templates, because your primary focus is on learning how to use these classes as a client.

Abstract Data Type (ADT)

- The atomic/primitive data types like **`bool`**, **`char`**, **`int`**, **`double`**, occupy the lowest level in the data structure hierarchy.
- To represent more complex information, you combine the atomic types to form larger structures.
- It is usually far more important to know how to use those structures effectively than to understand their underlying representation, e.g., using strings as abstract values.
- Abstract Data Type (ADT) is a conceptual model. They define what a data structure does without dictating how it does it. By separating the logical properties from the physical implementation, ADTs allow for cleaner, more maintainable code. It is a type defined in terms of its behavior rather than its representation.
- ADTs are central to the object-oriented style of programming, which encourages programmers to think about data structures in a holistic way.

Abstract Data Type (ADT)





Abstract Data Type (ADT)

Features of ADT

- Only the essentials are exposed, omitting implementation details (Abstraction).
- Data and its related operations are bundled into a single unit (Encapsulation).
- You can change the underlying implementation (e.g., from an Array to a Linked List) without affecting how the rest of the program uses the ADT (Data Independence).

Container class templates

Sequence containers:

array C++11	Array class (class template)
vector	Vector (class template)
deque	Double ended queue (class template)
forward_list C++11	Forward list (class template)
list	List (class template)

Container adaptors:

stack	LIFO stack (class template)
queue	FIFO queue (class template)
priority_queue	Priority queue (class template)

Associative containers:

set	Set (class template)
multiset	Multiple-key set (class template)
map	Map (class template)
multimap	Multiple-key map (class template)

Unordered associative containers:

unordered_set C++11	Unordered Set (class template)
unordered_multiset C++11	Unordered Multiset (class template)
unordered_map C++11	Unordered Map (class template)
unordered_multimap C++11	Unordered Multimap (class template)

Other:

Two class templates share certain properties with containers, and are sometimes classified with them: `bitset` and `valarray`.

General Guidelines for Using ADT Classes

- These classes represent ***ADTs*** whose details are hidden.
- Each class requires ***type*** parameters, e.g. vector<int>.
- Declaring variables of these types always invokes a class ***constructor***.
- Any memory for these objects is ***freed (automatically)*** when its declaration scope ends.
- Assigning one value to another, ***copies*** the entire structure.
- To avoid copying, these structures are usually passed ***by reference*** in function calls.

The `vector<`*type*`>` Class

- The **`vector`** class provides a facility similar to list in Python, and the underlying implementation is based on array in C++.

```
#include <vector.h>
```

- Besides including the appropriate library interface, as a client of the **`vector`** class, you are concerned with a different set of issues and need to answer the following questions:
  - How is it possible to specify the type of object (base/element type) contained in a **`vector`**?
  - How does one create an object that is an instance of the **`vector`** class (declaration)?
  - What methods exist in the **`vector`** class to implement its abstract behavior?

Declaring a **vector** object

- Instead of using the class name alone, the collection classes require a type parameter that specifies the element type. For example, **vector<int>** represents a vector of integers.

vector<type> vec; e.g `vector<int> vec;` Initializes an empty vector of the specified element type.
vector<type> vec(n); e.g `vector<int> vec(5);` Initializes a vector with **n** elements all set to the default value of the type.
vector<type> vec(n, value); e.g. `vector<int> vec(5,42);` Initializes a vector with **n** elements all set to **value**.
`vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9};//after C++ 11`

Declaring a `vector` object

- It is possible to create ***nested*** vectors, so that, for example, `vector< vector<char> >` represents a two-dimensional vector of characters.
- The spaces around the inner type parameter are necessary for many compilers to ensure that the angle brackets for the type parameters are interpreted correctly. Remember that `>>` is a single extraction operator.
- The type parameter used in the `vector` class can be any C++ type, so you could use the following definition to represent a three-dimensional vector (if a grid ADT has been implemented:

```
vector< grid<char> > chessPositions;
```

Methods in the STL `vector<`*type*`>` Class

fx Member functions

(constructor)	Construct vector (public
(destructor)	Vector destructor (public
operator=	Assign content (public m

Iterators:

begin	Return iterator to begin
end	Return iterator to end (
rbegin	Return reverse iterator t
rend	Return reverse iterator t
cbegin C++11	Return const_iterator to
cend C++11	Return const_iterator to
crbegin C++11	Return const_reverse_it
crend C++11	Return const_reverse_it

Capacity:

size	Return size (public memb
max_size	Return maximum size (
resize	Change size (public mem
capacity	Return size of allocated
empty	Test whether vector is e
reserve	Request a change in cap
shrink_to_fit C++11	Shrink to fit (public mem

Element access:

operator[]	Access element (public member function)
at	Access element (public member function)
front	Access first element (public member function)
back	Access last element (public member function)
data C++11	Access data (public member function)

Modifiers:

assign	Assign vector content (public member function)
push_back	Add element at the end (public member function)
pop_back	Delete last element (public member function)
insert	Insert elements (public member function)
erase	Erase elements (public member function)
swap	Swap content (public member function)
clear	Clear content (public member function)
emplace C++11	Construct and insert element (public member function)
emplace_back C++11	Construct and insert element at the end (public member function)

Allocator:

get_allocator	Get allocator (public member function)

fx Non-member function overloads

relational operators	Relational operators for vector (function template)
swap	Exchange contents of vectors (function template)

Template specializations

vector<bool>	Vector of bool (class template specialization)

The **vector<*type*>** Class Operators

```
#include <vector.h>

vector<int> vec;

vec.push_back(10);

vec.push_back(20);

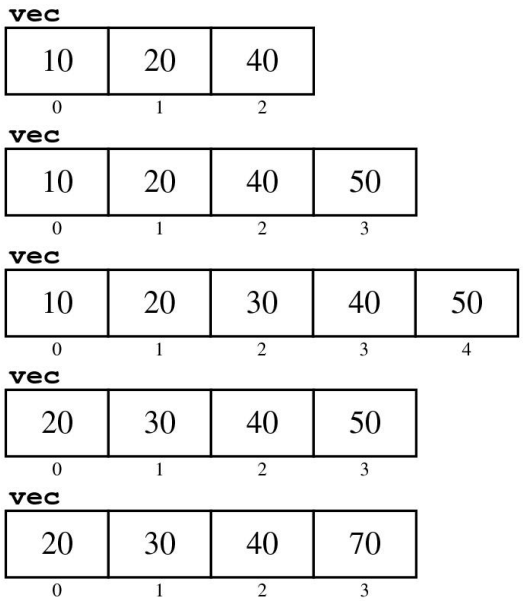
vec.push_back(40);

vec.push_back(50);

vec.insert(vec.begin()+2, 30);

vec.erase(vec.begin());

vec[3] = 70;
```



Vector.h

Exercise: `reverseEntireFile`

```
void ReverseEntireFile()
{
     ifstream ifile; // input file stream object
     ofstream ofile; // output file stream object
     vector<string> lines;
     string line;

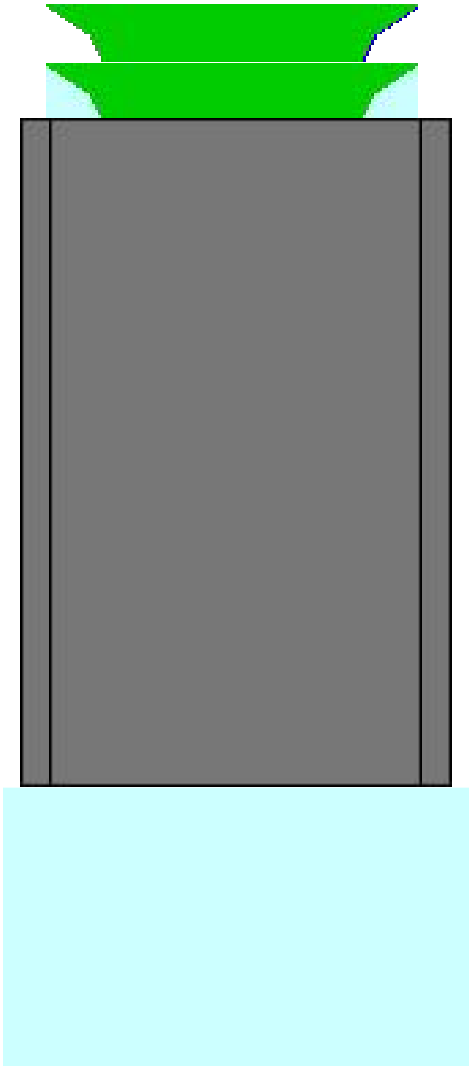
     ifile.open("input.txt");
     ofile.open("output.txt", std::ios::out | std::ios::beg);

     while (getline(ifile, line)) // read a line from input file
          lines.push_back(line);
     ifile.close();

     for (int i = lines.size(); i > 0; i--)
     {
          ofile << lines[i - 1] << endl; // write lines to output file
     }
     ofile.flush();
     ofile.close();
}
```

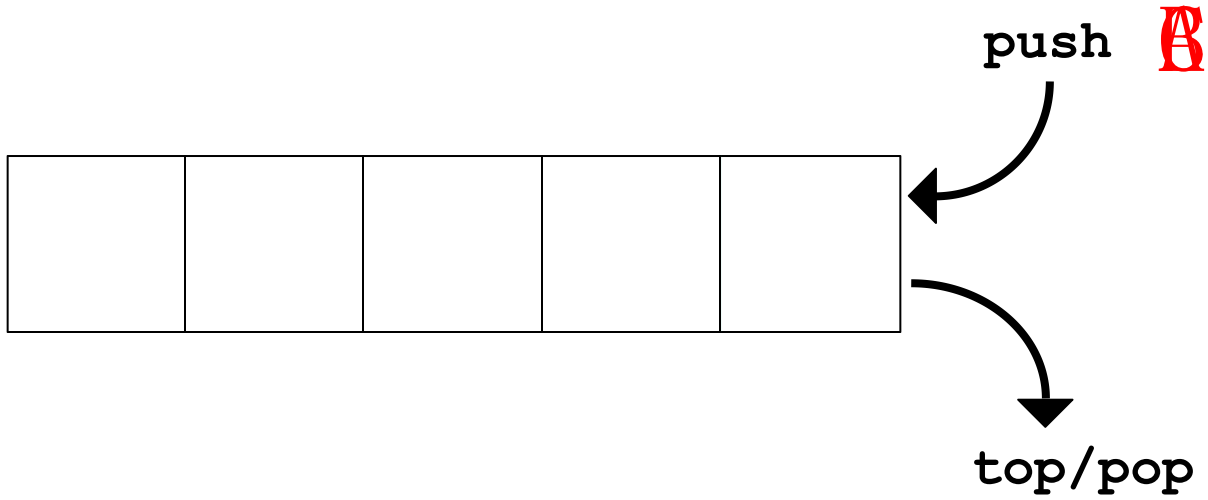
The Stack Metaphor

- A ***stack*** is a data structure in which the elements are accessible only in a ***Last In, First Out (LIFO)*** order.
- The fundamental operations on a stack are `push`, which adds a new value to the top of the stack, and `pop`, which removes and returns the top value.
- One of the most common metaphors for the stack concept is a spring-loaded storage tray for dishes. Adding a new dish to the stack pushes any previous dishes downward. Taking the top dish away allows the dishes to pop back up.
- In programming, nested function calls behave in a stack-oriented fashion.



Stacks

Stack:



Methods in the STL `stack<`*type*`>` Class

fx Member functions

(constructor)	Construct stack (public member function)
empty	Test whether container is empty (public member function)
size	Return size (public member function)
top	Access next element (public member function)
push	Insert element (public member function)
emplace C++11	Construct and insert element (public member function)
pop	Remove top element (public member function)
swap C++11	Swap contents (public member function)

fx Non-member function overloads

relational operators	Relational operators for stack (function)
swap (stack) C++11	Exchange contents of stacks (public member function)

fx Non-member class specializations

uses_allocator<stack> C++11	Uses allocator for stack (class template)

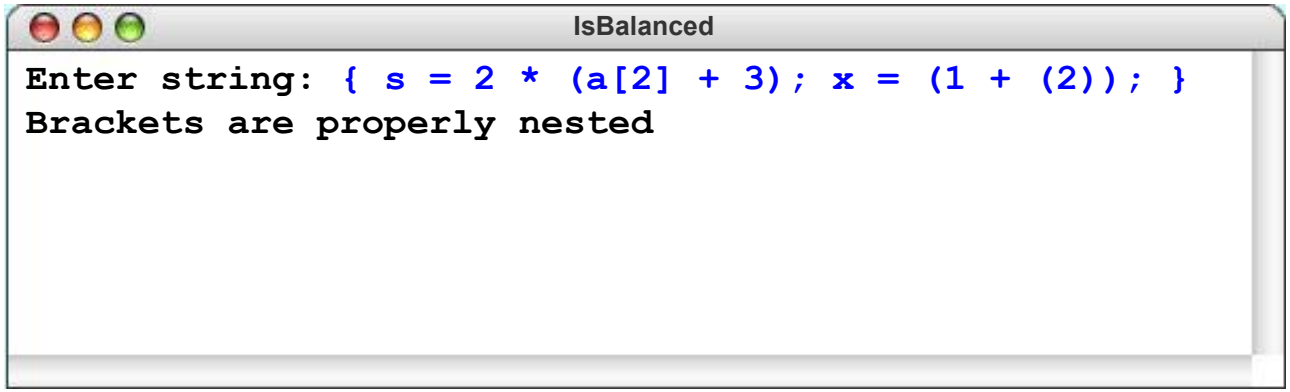
Stack.h

Example: Stack Processing

Write a C++ program that checks whether the bracketing operators (parentheses, brackets, and curly braces) in a string are properly matched. As an example of proper matching, consider the string

```
{ s = 2 * (a[2] + 3); x = (1 + (2)); }
```

If you go through the string carefully, you discover that all the bracketing operators are correctly nested, with each open parenthesis matched by a close parenthesis, each open bracket matched by a close bracket, and so on.



```
void TestBalance()
{
	string str;
	cout << "Enter a string:";
	cin >> str;

	if (isBalanced(str))
	cout << "Brackets are properly nested" << endl;
	else
	cout << "Brackets are incorrect" << endl;
}



bool operatorMatches(char open, char close)
{
	switch (open)
	{
		case '{':
		return close == '}';
		case '[':
		return close == ']';
		case '(':
		return close == ')';
		default:
		return false;
	}
}
```

```
bool isBalanced(string str)
{
	Stack<char> stack;
	for (int i = 0; i < str.length(); i++)
	{
		char ch = str[i];
		switch (ch)
		{
			case '{':
			case '[':
			case '(':
			stack.push(ch);
			break;
			case '}':
			case ']':
			case ')':
			if (stack.empty())
			return false;
			char c = stack.top();
			stack.pop();
			if (!operatorMatches(c, ch))
			return false;
			break;
		}
	}
return stack.empty();
}
```

Iterating over a collection

- One of the common operations that clients need to perform when using a collection is to iterate through the elements.
- While it is easy to implement iteration for vectors and grids using **`for`** loops, it is less clear how you would do the same for other collection types. The modern approach to solving this problem is to use a general tool called an ***iterator*** that delivers the elements of the collection, one at a time.
- C++11 uses a ***range-based for statement*** to simplify iterators:

```
for (string key : map) { . . . code to process that key . . . }
```

- ***Range-based for*** (provided since C++ 11) is a way to access ***iterators*** (provided by C++ and implemented by the implementors of the collections).
- The dereference operator (*), pre-inclement operator (++) and inequality operator (!=) are required for a range-based iterator.

Iterator Order

- When you look at the documentation for an iterator, one of the important things to determine is whether the collection class specifies the order in which elements are generated. e.g.
  - Iterators for the **`vector`** class operate in index order.
  - Iterators for the **`grid (matrix)`** class operate in ***row-major order***, which means that the iterator runs through every element in row 0, then every element in row 1, and so on.
  - Iterators for the **`map`** class deliver the keys in the order imposed by the standard comparison function for the key type;
  - Iterators for the **`Set`** class deliver the elements in the order imposed by the standard comparison function for the value type;
  - Iterators for the **`Binary Search Tree`** class can operates in the order of in-order traversal.

Forward Iterator

```
class iterator
{
  private:
    T *ptr;
  public:
    iterator(T *p) : ptr(p) {} // constructor

    T &operator*() const // Dereference operator
    {
            return *ptr;
    }

    iterator &operator++() // Pre-increment operator
    {
       ++ptr;
       return *this;
    }

    bool operator!=(const iterator &other) const // Inequality comparison
    {
       return ptr != other.ptr;
     }
.
. (more operators/functions
.
};
```

Implementation of a simple forward iterator

Forward Iterator

```
// Return an iterator to the beginning
iterator begin() const
{
    iterator it = iterator(data);
    return it;
}

// Return an iterator to the end
iterator end() const
{
    iterator it = iterator(data + count);
    return it;
}
```

Functions in ATD

```
cout << "forward iterator:";
for (Stack<int>::iterator it = cs.begin(); it != cs.end(); it++)
cout << *it << " ";
cout << endl;

for (auto x : cs) //range-based
cout << x << " ";
cout << endl;
```

Use the iterator

Reverse Iterator

```
class reverse_iterator
{
private:
   T *ptr;

public:
   reverse_iterator(T *p) : ptr(p) {} // constructor

T &operator*() const // Dereference operator
{
   T *tmp = ptr;
   return *(--tmp);
}

reverse_iterator &operator++() // Pre-increment operator
{
  --ptr;
  return *this;
}

// Inequality comparison
bool operator!=(const reverse_iterator &other) const
{
   return ptr != other.ptr;
}
};
```

Implementation of a simple reverse iterator

Reverse Iterator

```
reverse_iterator rbegin() const
{
   reverse_iterator it = reverse_iterator(data + count);
   return it;
}

reverse_iterator rend() const
{
   reverse_iterator it = reverse_iterator(data);
   return it;
}
```

Functions in ATD

```
cout << "backward iterator:";
for (Stack<int>::reverse_iterator it = cs.rbegin(); it != cs.rend(); ++it)
{
            cout << *it << " ";
}
cout << endl;
```

Use the reverse-iterator

The End